Practical No: 01

Aim: Design and implement basics embedded circuits

1. Automatic Alarm system- Alarm should get trigger by senor

2. Timer based buzzer

3. Sensor based Counting device.

Design an automatic alarm system where an alarm (buzzer) is activated whenever a sensor detects an object or motion.

Components Required

- Arduino Uno
- PIR Motion Sensor (or IR Sensor)
- Buzzer

Circuit Connections

Component	Arduino Pin
PIR VCC	5V
PIR GND	GND
PIR OUT	D2
Buzzer (+)	D8
Buzzer (-)	GND

Code:

```
const int sensor = 2;
const int buzzer = 5;

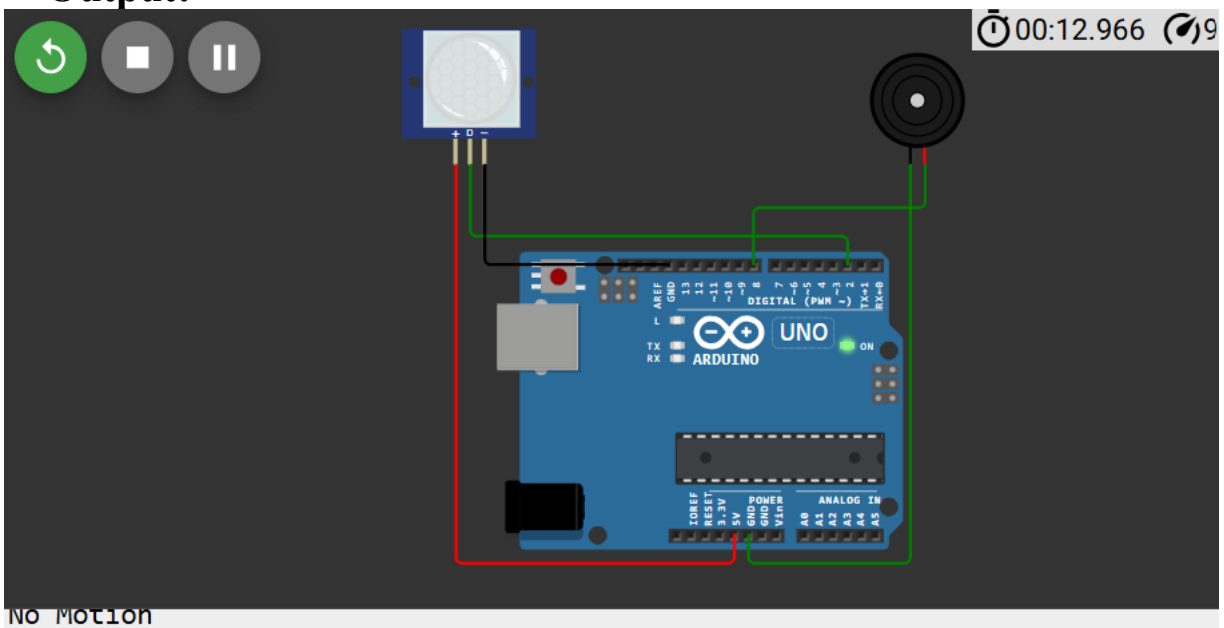
void setup() {
  pinMode(sensor, INPUT);
  pinMode(buzzer, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int sensorState = digitalRead(sensor);

  if (sensorState == HIGH) {
    digitalWrite(buzzer, HIGH);
    Serial.println("Motion Detected!");
  } else {
    digitalWrite(buzzer, LOW);
    Serial.println("No Motion");
  }

  delay(200);
}
```

Output:




```
No Motion
No Motion
Motion Detected!
Motion Detected!
```

Practical No: 02

Aim: Demonstrate communication between two embedded devices using UART port.

Components Required :

Sr. No.	Component	Quantity
1	Arduino Uno	2

Connection:

Sender (Arduino1)	**Receiver (Arduino 2)**
TX (Pin 1)	RX (Pin 0)
RX (Pin 0)	TX (Pin 1) *(For two-way comm.)*
GND	GND

Code:

```
void setup() {
  Serial.begin(9600);       // Start UART communication
}

void loop() {

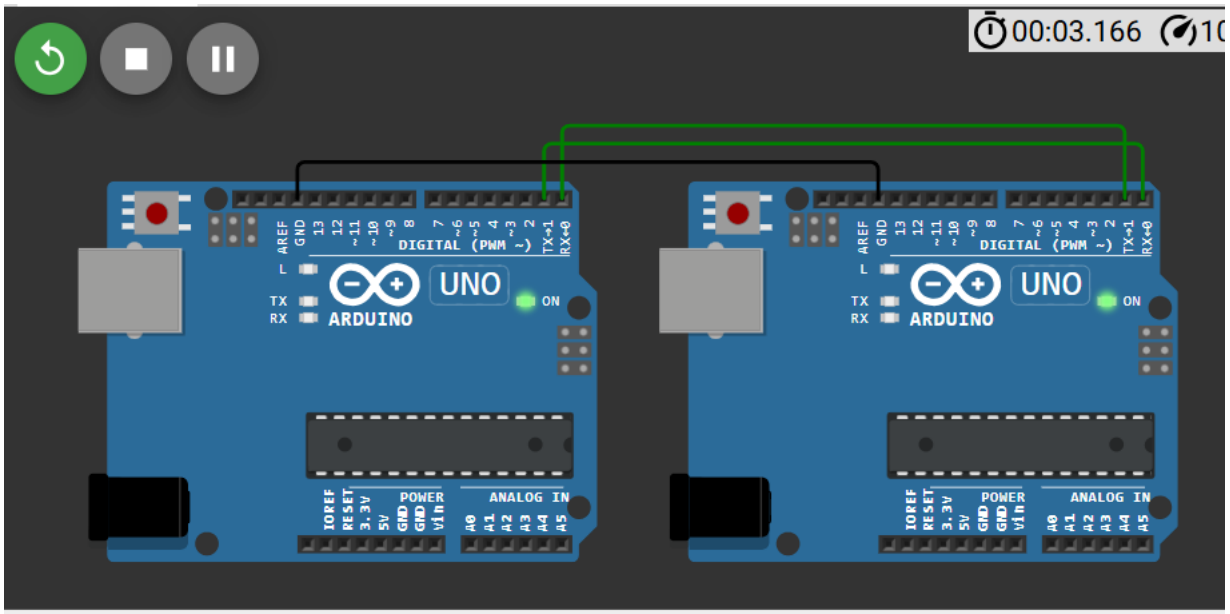
  // Sender
  Serial.println("Hello from Arduino");

  // Receiver
  while (Serial.available() > 0) {
    String msg = Serial.readStringUntil('\n');
    Serial.print("Received: ");
    Serial.println(msg);
```

```
  }

  delay(2000);      // Wait for 2 seconds
}
```

Output:




```
Hello from Arduino
Hello from Arduino
```

Practical No:03

Aim: Built an IoT system to send ticket before entering the bus.

Components:

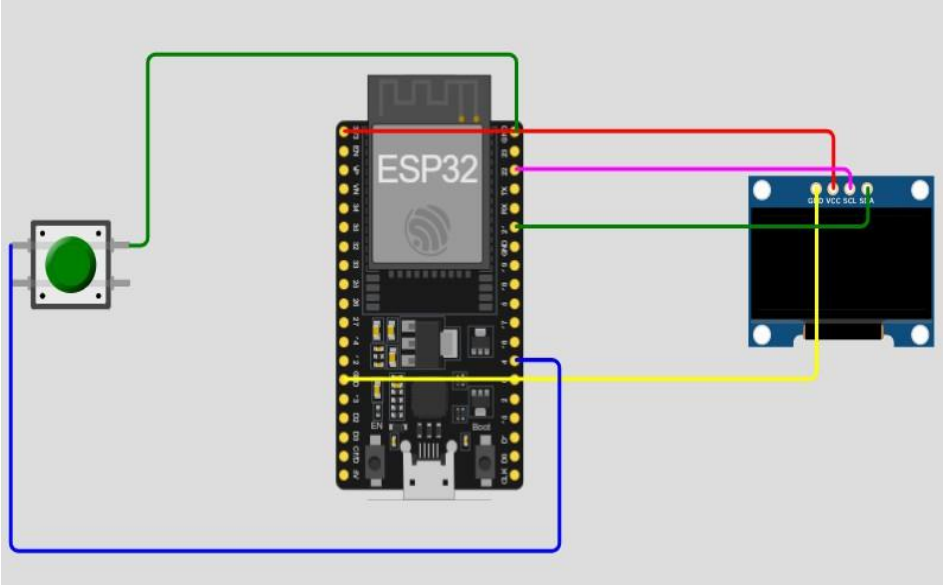
- ESP32
- SSD1306 OLED Display (128×64, I2C)
- Push Button

Connections

OLED Display

OLED Pin	ESP32 Pin
VCC	3.3V
GND	GND
SDA	GPIO 21
SCL	GPIO 22

Button Pin	ESP32 Pin
One Side	GPIO 4
Other Side	GND



Code:

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire,
OLED_RESET);

const int buttonPin = 4;
int ticketNumber = 1000;
// Today's Date
String currentDate = "01/08/2026";
void setup() {
Serial.begin(115200);
  pinMode(buttonPin, INPUT_PULLUP);
  Wire.begin(21,22);
  if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("OLED Failed");
    while(true);
  }

  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);

  display.setCursor(10,10);
  display.println("SMART BUS");

  display.setCursor(10,30);
  display.println("Ticket System");

  display.display();
```

```
  delay(2000);
}

void loop() {

  if(digitalRead(buttonPin) == LOW) {
    ticketNumber++;
    Serial.println("--------------------------");
    Serial.println("SMART BUS TICKET");
    Serial.print("Date      : ");
    Serial.println(currentDate);
    Serial.print("Ticket ID : BUS");
    Serial.println(ticketNumber);
    Serial.println("Status    : Ticket Sent");

    display.clearDisplay();
    display.setTextSize(1);

    display.setCursor(0,0);
    display.println("SMART BUS");

    display.print("Date:");
    display.println(currentDate);

    display.print("Ticket:");
    display.println(ticketNumber);

    display.println("Status:");
    display.println("Ticket Sent");

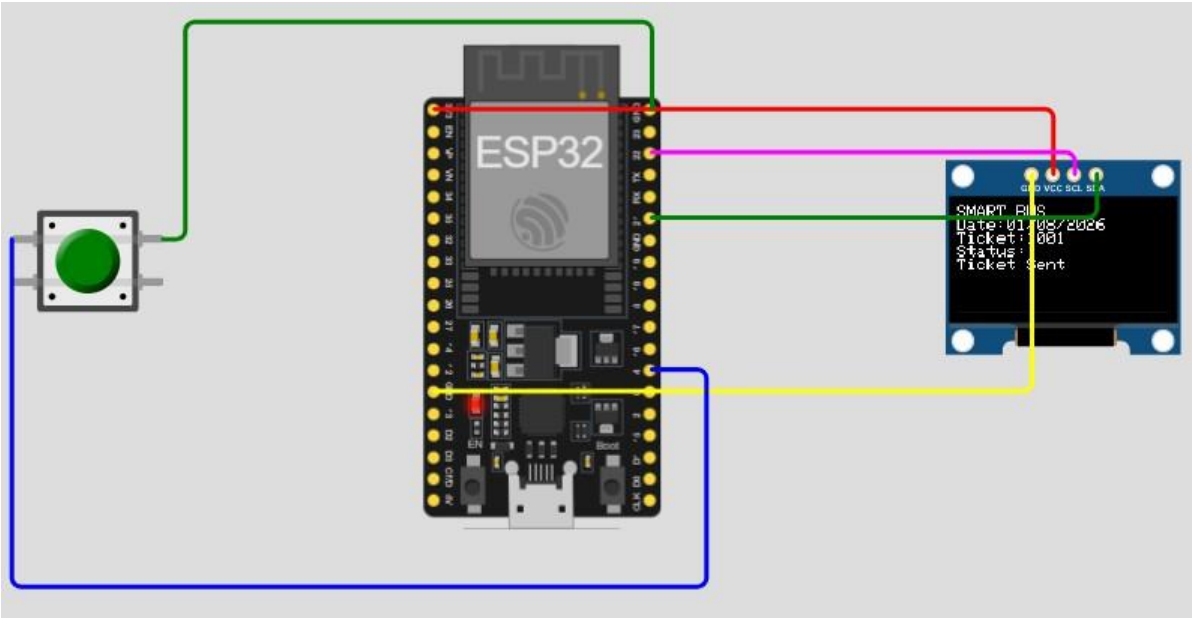
    display.display();
    delay(1000);
    while(digitalRead(buttonPin)==LOW);
  }
}
```

You need to add the libraries to your project.

1. Open your Wokwi project.
2. Open the Library Manager (or edit libraries.txt if your project has one).
3. Add these two libraries:

Adafruit GFX Library
Adafruit SSD1306

Output:



Practical No: 04

Aim: Demonstrate an IoT based game which can be played between two player who are physically at a considerable distance.

Components:

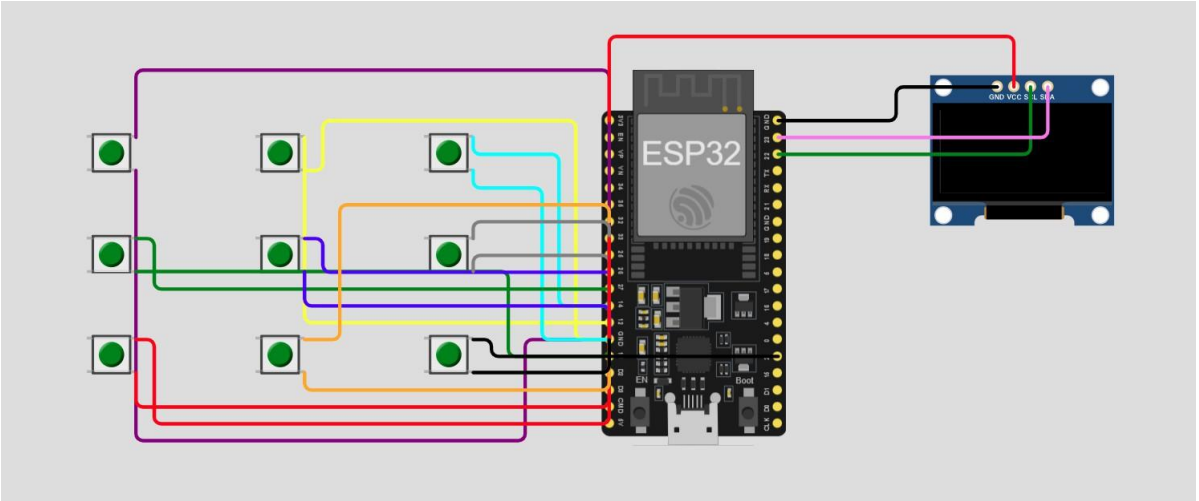
1. ESP32
2. Buttons(9)
3. SSD1306 OLED Display

Connection: Buttons To ESP32

Button	**Tic-Tac-Toe position**	**ESP32 GPIO**	**Other side**
Button 1	1	**13**	GND
Button 2	2	**12**	GND
Button 3	3	**14**	GND
Button 4	4	**27**	GND
Button 5	5	**26**	GND
Button 6	6	**25**	GND
Button 7	7	**33**	GND
Button 8	8	**32**	GND
Button 9	9	**2**	GND

Connection : SSD1306 OLED Display To ESP32

OLED pin	**ESP32**
GND	GND
VCC	3V3
SCL	GPIO 22
SDA	GPIO 23



Code:

```
#include <Wire.h>
#include
<Adafruit_GFX.h>
#include
<Adafruit_SSD1306.h>
#define
SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

// Tic-Tac-Toe
buttons int
buttons[9] = {
 13, 12, 14,
 27, 26, 25,
 33, 32, 2
};
// Game
board char
board[9] = {
' ', ' ', ' ',
 ' ', ' ', ' ',
 ' ', ' ', ' '
};
```

```
bool playerX =
true; bool
gameOver =
false;
/ ============================
// SETUP
// ============================

void setup() {

 Serial.begin(115200);

 // Buttons
 for (int i = 0; i < 9; i++) {
  pinMode(buttons[i],
  INPUT_PULLUP);
 }

 // OLED
 if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
  Serial.println("OLED initialization
  failed!"); while (1);
 }
 display.clearDisplay();
 display.setTextColor(SSD1306_
 WHITE);
 Serial.println();
 Serial.println("============================
 ==");
 Serial.println(" IoT Tic-Tac-Toe Started");
 Serial.println("================================");

 Serial.println("Player X starts!");

 showBoard();
}
```

```
// ============================
// LOOP
// ============================

void loop() {

 if (gameOver)
  { return;
 for (int i = 0; i < 9; i++) {

  if (digitalRead(buttons[i]) ==
   LOW) { delay(200);
   // Check empty
   position if
   (board[i] == ' ') {

    // Player X
    or O if
    (playerX) {
    board[i] =
    'X';

     Serial.print("Player X selected position ");
     Serial.println(i + 1);


    } else {
     board[i] =
     'O';

     Serial.print("Player O selected position ");
     Serial.println(i + 1);
    }

    // Update
    OLED
    showBoard();

    // Check winner
```

```
if
 (checkWinner())
 { char winner;
 if (playerX)
  { winner =
  'X';
 } else {
  winner =
  'O';
 }

 showWinner(winner);

 Serial.println();
 Serial.println("==========================
 =====");
 Serial.print("
                 PLAYE
 R ");
 Serial.print(winner);
 Serial.println("
 WINS!");
 Serial.println("================================");

 gameOver = true;

 return;
}
// Check draw
if (checkDraw()) {

 Serial.println();
 Serial.println("==========================
 =====");
 Serial.println("  DRAW!");
 Serial.println("================================");

 display.clearDisplay();
 display.setTextSize(2);
```

```
        display.setCursor(25, 25);
        display.println("DRAW!");
        display.display();

        gameOver = true;

        return;
       }
       // Change player
       playerX =
       !playerX;

       if (playerX) {
        Serial.println("Now Player X's turn");
       } else {
        Serial.println("Now Player O's turn");
       }

      } else {

       Serial.print("Position ");
       Serial.print(i + 1);
       Serial.println(" already occupied!");
      }

      // Wait until button released
      while (digitalRead(buttons[i])
       == LOW) { delay(10);
      }

      delay(100);
     }
    }
}
```

```
// ============================
// SHOW BOARD
// ============================

void showBoard() {

 Serial.println();
 Serial.println(" TIC-TAC-TOE");

 Serial.print("");
 Serial.print(board
 [0]);
 Serial.print(" | ");
 Serial.print(board
 [1]);
 Serial.print(" | ");
 Serial.println(boa
 rd[2]);

 Serial.println(" ---+---+---");


 Serial.print("");
 Serial.print(board
 [3]);
 Serial.print(" | ");
 Serial.print(board
 [4]);
 Serial.print(" | ");
 Serial.println(boa
 rd[5]);

 Serial.println(" ---+---+---");


 Serial.print("");
 Serial.print(board[6]);
 Serial.print(" | ");
 Serial.print(board[7]);
 Serial.print(" | ");
 Serial.println(board[8]);
```

```
Serial.println();



// OLED
display.clearDis
play();

display.setTextSize(1);
display.setCursor(35, 0);
display.println("TIC-
TAC-TOE");

display.setTextSize(2);


// Row 1
display.setCursor(10, 15);
display.print(board[0]);

display.setCursor(58, 15);
display.print(board[1]);

display.setCursor(106, 15);
display.print(board[2]);

// Row 2
display.setCursor(10, 32);
display.print(board[3]);

display.setCursor(58, 32);
display.print(board[4]);

display.setCursor(106, 32);
display.print(board[5]);

// Row 3
display.setCursor(10, 49);
display.print(board[6]);
display.setCursor(58, 49);
display.print(board[7]);
```

```
 display.setCursor(106, 49);
 display.print(board[8]);

 display.display();
}

============================
// CHECK WINNER
// ============================

bool
 checkWinner() {
 int win[8][3] = {
  {0, 1, 2},
  {3, 4, 5},
  {6, 7, 8},
  {0, 3, 6},
  {1, 4, 7},
  {2, 5, 8},
  {0, 4, 8},
  {2, 4, 6}
 };

 for (int i = 0; i < 8;
  i++) { if (
   board[win[i][0]] != ' ' &&
   board[win[i][0]] == board[win[i][1]] &&
   board[win[i][1]] == board[win[i][2]]
  ) {

   return true;
  }
 }
 return false;
}

// ============================
```

```
// CHECK DRAW
// ============================
bool checkDraw() {

 for (int i = 0; i < 9; i++) {

  if (board[i] == '
   ') { return
   false;
  }
 }

 return true;
}



// ============================
// SHOW WINNER ON OLED
// ============================

void showWinner(char winner) {
 display.clearDisplay();
 display.setTextSize(2);
 display.setCursor(15, 10);
 display.println("PLAYER");

 display.setCursor(55, 32);
 display.print(winner);

 display.setCursor(7
 0,                  32);
 display.println("WI
 NS!");
 display.display();
}
```

Add Libraries : Add this 2 libraries in library manager

Adafruit GFX Library

Adafruit SSD1306

Output:

```
ets Jul 29 2019 12:21:46

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc

================================
 IoT Tic-Tac-Toe Started
================================
Player X starts!

      TIC-TAC-TOE
          |   |
       ---+---+---
          |   |
       ---+---+---
          |   |

Player X selected position 1

      TIC-TAC-TOE
        X |   |
       ---+---+---
          |   |
       ---+---+---
          |   |
```

```
Now Player X's turn
Player X selected position 9

        TIC-TAC-TOE
          X |   |
         ---+---+---
          O | O |
         ---+---+---
          X |   | X

Now Player O's turn
Player O selected position 6

        TIC-TAC-TOE
          X |   |
         ---+---+---
          O | O | O
         ---+---+---
          X |   | X


==================================
        PLAYER O WINS!
==================================
```

```
Now Player O's turn
Player O selected position 5

        TIC-TAC-TOE
          X |   |
         ---+---+---
            | O |
         ---+---+---
            |   |

Now Player X's turn
Player X selected position 7

        TIC-TAC-TOE
          X |   |
         ---+---+---
            | O |
         ---+---+---
          X |   |

Now Player O's turn
Player O selected position 4

        TIC-TAC-TOE
          X |   |
         ---+---+---
          O | O |
         ---+---+---
          X |   |
```

Practical No: 05

Aim : Develop a IoT application which will record the movement and orientation of your phone and give the data back to the PC.

Components
- Component Quantity
- Android smartphone 1
- PC/Laptop 1
- Wi-Fi network / mobile hotspot 1
- USB cable 1
- Arduino/ESP32 (optional) 1
- Arduino IDE / Python 1

MPU6050 → ESP32 pin connection

MPU6050 pin ESP32 pin Connection
VCC 3.3V Power
GND GND Ground
SDA GPIO 21 I2C data
SCL GPIO 22 I2C clock
INT Not required Optional

Android Code
A practical Android implementation can use the SensorManager API to read the accelerometer and gyroscope.

```
SensorManager sensorManager;
Sensor accelerometer;
Sensor gyroscope;
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
sensorManager =
(SensorManager) getSystemService(Context.SENSOR_SERVICE);
accelerometer =
sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER);
gyroscope =
sensorManager.getDefaultSensor(Sensor.TYPE_GYROSCOPE);
}
@Override
protected void onResume() {
```

```
super.onResume();
sensorManager.registerListener(
this,
accelerometer,
SensorManager.SENSOR_DELAY_NORMAL
);
sensorManager.registerListener(
this,
gyroscope,
SensorManager.SENSOR_DELAY_NORMAL
);
}
@Override
protected void onPause() {

super.onPause();
sensorManager.unregisterListener(this);
}
```

PC Python Code

The following simple Python program acts as a TCP receiver and displays the incoming phone data.

```
import socket
HOST = "0.0.0.0"
PORT = 5000
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(1)
print("Waiting for Android phone...")
print("Listening on port:", PORT)
connection, address = server.accept()
print("Phone connected:", address)
print("--------------------------------")
while True:
data = connection.recv(1024)
if not data:
break
message = data.decode("utf-8")
print(message)
connection.close()
```

server.close()
Run it on the PC with:
python server.py

Serial Monitor / Output:

```
==========================================
PHONE MOTION MONITOR
==========================================
Accelerometer:
X = 0.24 m/s2
Y = 9.61 m/s2
Z = 0.87 m/s2
Gyroscope:
X = 0.012 rad/s
Y = -0.034 rad/s
Z = 0.008 rad/s
Orientation:
Roll = 2.45°
Pitch = 5.32°
Yaw = 91.74°
Status: PHONE MOVING
WiFi: CONNECTED
----------------------------------------
Accelerometer:
X = 1.82 m/s2
Y = 8.94 m/s2
Z = 1.21 m/s2
Gyroscope:
X = 0.145 rad/s
Y = 0.092 rad/s
Z = -0.031 rad/s
Orientation:
Roll = 7.21°
Pitch = 11.48°
Yaw = 94.16°
Status: PHONE ROTATING
WiFi: CONNECTED
----------------------------------------
```

Practical No: 06

Aim : Develop an IoT application that will raise an alarm whenever with going to rain outside based on the weather prediction data.

Components :

- ESP32
- LED
- Resistor(220 ohm)
- Buzzer

Connections:
Resistor To
ESP32
Resisto
r 1 side

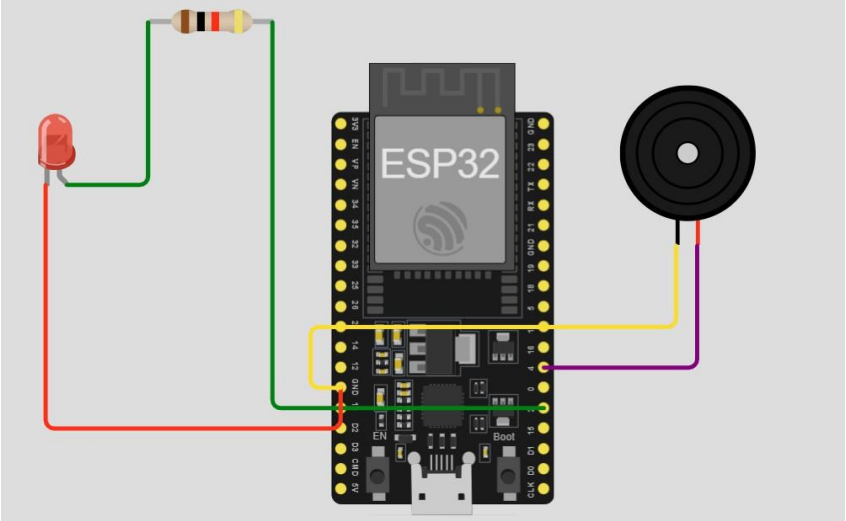
GPIO 2

Resistor To LED

Resistor
other side

LED(1
side) LED
To ESP32
LED
other
side

ESP32 (GND)
Buzzer To ESP32
Buzzer 1 side
GND
Buzzer other side
GPIO 4 Add
Libraries :
ArduinoJson



Code:

```
#include <WiFi.h>
#include
<HTTPClient.h>
#include
<WiFiClientSecure.h>
#include <ArduinoJson.h>

//---------------------------------
// Hardware pins
//---------------------------------
const int RED_LED
= 2; const int
BUZZER = 4;

//---------------------------------
// WiFi settings for Wokwi
//---------------------------------
const char* WIFI_SSID = "Wokwi-
GUEST"; const char*
WIFI_PASSWORD = "";

//---------------------------------
// Rain alarm threshold
//---------------------------------
```

```
const int rainThreshold = 60;

//---------------------------------
// Alarm settings
//---------------------------------
const int alarmFrequency = 2500;
// Hz const int alarmDuration =
4000; // 4 seconds

//---------------------------------
// Weather location: Mumbai
//---------------------------------
const char* weatherURL =
"https://api.open-
meteo.com/v1/forecast"
"?latitude=19.0760"
"&longitude=72.8777"
"&hourly=precipitation_probabi
lity" "&forecast_hours=1"
"&timezone=auto";

// ========================================
// SETUP
// ========================================
void setup() {

Serial.begin(115200);

pinMode(RED_LED,
OUTPUT);
pinMode(BUZZER,
OUTPUT);

digitalWrite(RED_LED, LOW);
digitalWrite(BUZZER, LOW);

Serial.println();
Serial.println("==============================
===");
Serial.println(" IoT RAIN PREDICTION SYSTEM");
```

```
Serial.println("==================================");

// Connect WiFi
Serial.print("Connecting to
WiFi");


WiFi.begin(WIFI_SSID, WIFI_PASSWORD, 6);

while (WiFi.status() !=
WL_CONNECTED) { delay(500);
Serial.print(".");
}

Serial.println();
Serial.println("WiFi
Connected!");

Serial.print("ESP32 IP Address:
"); Serial.println(WiFi.localIP());

// Get weather
data
getWeatherData()
;
}

// ========================================
// RAIN ALARM
// ========================================
void rainAlarm() {

Serial.println();
Serial.println("************************
******"); Serial.println(" !!! RAIN
WARNING !!!"); Serial.println(" ALARM
ON FOR 4 SEC");
Serial.println("************************
******");

// Red LED ON
```

```
digitalWrite(RED_LED,
HIGH);

// Loud continuous buzzer for 4
seconds tone(BUZZER,
alarmFrequency);

delay(alarmDuration);

// Stop buzzer
noTone(BUZZ
ER);

// Keep LED ON to indicate rain
warning digitalWrite(RED_LED,
HIGH);
}

// ========================================
// GET WEATHER DATA
// ========================================
void getWeatherData() {

Serial.println();
Serial.println("Getting weather prediction...");

WiFiClientSecure client;
client.setInsecure();

HTTPClient http;
http.begin(client,
weatherURL); int
httpCode = http.GET();
if (httpCode > 0) {

Serial.print("HTTP Response
Code: ");
Serial.println(httpCode);

if (httpCode == HTTP_CODE_OK) {
```

```
String payload = http.getString();
Serial.println("Weather data
received!");
// JSON
document
JsonDocument
doc;

DeserializationError
error =
deserializeJson(doc,
payload);

if (error) {
Serial.print("JSON Error: ");
Serial.println(error.c_str());

http.end();
return;
}

// Get rain
probability int
rainProbability =
doc["hourly"]["precipitation_probability"][0];

Serial.print("Rain Probability: ");
Serial.print(rainProbability);
Serial.println("%");

// ==================================
// CHECK RAIN CONDITION
// ==================================
if (rainProbability >=
rainThreshold) { Serial.println();
Serial.println("WARNING: RAIN
EXPECTED!"); rainAlarm();
} else {

Serial.println("No significant rain expected.");
Serial.println("Alarm OFF")
```

```
digitalWrite(RED_LED, LOW);
noTone(BUZZER);
}
}
} else {

Serial.print("HTTP request failed: ");
Serial.println(http.errorToString(httpCode));
}

http.end();
}
// ========================================
// LOOP
// ========================================
void loop() {

// Check weather every 5 minutes
delay(300000);

getWeatherData();
}
```

Output:

```
================================
 IoT RAIN PREDICTION SYSTEM
================================
Connecting to WiFi......
WiFi Connected!
ESP32 IP Address: 10.10.0.2

Getting weather prediction...
HTTP Response Code: 200
Weather data received!
Rain Probability: 100%

WARNING: RAIN EXPECTED!

**********************************
 !!! RAIN WARNING !!!
 ALARM ON FOR 4 SEC
**********************************
```

Practical No: 07

Aim: Deploy an IoT application which will alert you by beeping or vibrating your Page 35 of 48 phone whenever you get someone call your name.

Components in Wokwi

- ESP32
- Push Button
- LED
- 220Ω Resistor
- Buzzer
- OLED 128×64

Connection:

Component
ESP32 Button pin
1 GPIO 18
Button pin 2
GND
LED Anode GPIO 2 through

220Ω
LED Cathode
GND Buzzer +
GPIO 15
Buzzer − GND
OLED VCC
3.3V

OLED GND
GND OLED
SDA GPIO 21
OLED SCL
GPIO 22

Add Library :
Adafruit
SSD1306
Adafruit GFX
Library

Sketch.ino code:

```
#include <Wire.h>
#include
<Adafruit_GFX.h>
#include
<Adafruit_SSD1306.h>

#define BUTTON_PIN 18
#define LED_PIN 2
#define BUZZER_PIN 15

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

Adafruit_SSD1306 display(SCREEN_WIDTH,
SCREEN_HEIGHT, &Wire, -1);
void setup() {
Serial.begin(1152
00);
pinMode(BUTTON_PIN,
INPUT_PULLUP); pinMode(LED_PIN,
OUTPUT); pinMode(BUZZER_PIN,
OUTPUT);

digitalWrite(LED_PIN, LOW);
digitalWrite(BUZZER_PIN,
LOW);

// OLED start
if (!display.begin(SSD1306_SWITCHCAPVCC,
0x3C)) { Serial.println("OLED ERROR");
while (1);
}

display.clearDisplay();

display.setTextSize(1);
display.setTextColor(SSD1306_
WHITE);
```

```
display.setCursor(10, 10);
display.println("IoT NAME
ALERT");

display.setCursor(10, 30);
display.println("System Ready");
display.display();
Serial.println("=========================
==");
Serial.println(" IoT NAME ALERT SYSTEM");
Serial.println("==========================");
Serial.println("System Ready");
Serial.println("Press button when name
is called");
}

void loop() {

int buttonState =
digitalRead(BUTTON_PIN); if
(buttonState == LOW) {
Serial.println("NAME
DETECTED!");
Serial.println("ALERT
ACTIVATED!");

// LED ON
digitalWrite(LED_PIN, HIGH);

// Buzzer ON
digitalWrite(BUZZER_PIN,
HIGH);

// OLED alert
display.clearDispl
ay();

display.setTextSize(2);
display.setCursor(5, 5);
display.println("ALERT!");
```

```
display.setTextSize(1);
display.setCursor(5, 35);
display.println("NAME
DETECTED");

display.setCursor(5, 50);
display.println("VIBRATION
ALERT");

display.display();
delay(3000);
// OFF
digitalWrite(LED_PIN, LOW);
digitalWrite(BUZZER_PIN,
LOW);

display.clearDisplay();

display.setTextSize(1);
display.setCursor(10, 20);
display.println("System Ready");

display.display();

Serial.println("Alert
OFF"); delay(1000);
}
}
```

Diagram.json code :

```
{
"version": 1,
"author": "IoT Name
Alert", "editor":
"wokwi",
"parts": [
```

{ "type": "wokwi-esp32-devkit-v1", "id": "esp",
"top": 52.7, "left": -139.4, "attrs": {} },
{
"type": "wokwi-
pushbutton", "id":
"button",
"top": 73.4,
"left": 96,
"attrs": { "color": "green", "label": "NAME

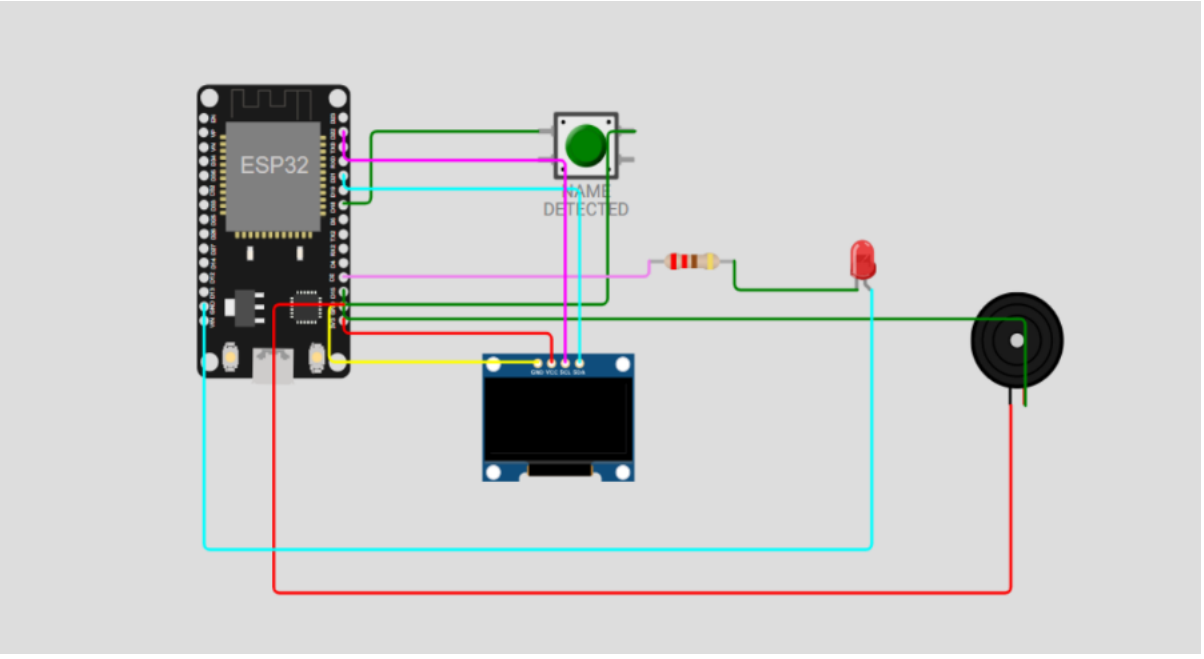
DETECTED" }
},
{ "type": "wokwi-led", "id": "led", "top": 150,
"left": 301.4, "attrs": { "color": "red" } },
{
"type": "wokwi-
resistor", "id":
"resistor",
"top": 167.15,
"left": 172.8,
"attrs": { "value": "220" }
},
{ "type": "wokwi-buzzer", "id": "buzzer", "top":
184.8, "left": 395.4, "attrs": {} },

{
"type": "board-ssd1306",
"id": "oled",
"top": 233.54,
"left": 57.83,
"attrs": { "i2cAddress": "0x3C" }
}
],
"connections": [
["button:1", "esp:18", "green", []],
["button:2", "esp:GND.1", "black", []],
["esp:2", "resistor:1", "green", []],
["esp:15", "buzzer:1", "green", []],
["oled:SDA", "esp:21", "green", []],
["oled:SCL", "esp:22", "green", []],

["button:1.l", "esp:D18", "green", [
"h-115.2", "v48"]],
["button:1.r", "esp:GND.1", "green", [
"h-19", "v115.2", "h-182.1"]],
["resistor:2", "led:C", "green", ["v0"]],
["esp:D2", "resistor:1", "violet", ["h0"]],
["led:A", "esp:GND.2", "purple",
["v28.8", "h-460.8"]],
["buzzer:2", "esp:D15", "green", ["v0"]],
["buzzer:1", "esp:GND.2", "green", ["v67.2",
"h-585.6", "v-134.3"]],
["oled:VCC", "esp:3V3", "red", ["v0"]],
["oled:GND", "esp:GND.2",
"yellow", ["v19.2", "h-240", "v-57.5"
]],

["oled:SDA", "esp:D21", "cyan", [
"v-115.2", "h-162.83"]],
["oled:SCL", "esp:D22", "magenta", [
"v-134.4", "h-153"]]
],
"serialMonitor": { "display":
"always" }, "dependencies": {}
}

Output:



Practical No: 08

Aim: Develop an IoT application for monitoring water levels in tanks and automatically start the motor to fill the tank if the level goes below the critical level.

Create a new **ESP32** project and add:

1. ESP32
2. Slide potentiometer
3. Relay Module
4. Red LED
5. 220Ω resistors
6. Buzzer

Connections

Slide Potentiometer → ESP32

HC-SR04	ESP32
VCC	3v3
GND	34
SIG	GND

Relay → ESP32

Relay	ESP32
VCC	5V
GND	GND
IN	GPIO 23

Red LED

LED	Connection
Anode (+)	GPIO 4 through 220Ω
Cathode (-)	GND

ESP32 to Buzzer

ESP32	**Buzzer**
ESP32	+ve
GND	other

Code:

sketch.inocode :

```
#define WATER_SENSOR_PIN 34
#define RELAY_PIN 23
#define LED_PIN 2
#define BUZZER_PIN 21

#define LOW_LEVEL 30
#define FULL_LEVEL 80

void setup() {
  Serial.begin(115200);

  pinMode(RELAY_PIN, OUTPUT);
  pinMode(LED_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  digitalWrite(RELAY_PIN, LOW);
  digitalWrite(LED_PIN, LOW);
  digitalWrite(BUZZER_PIN, LOW);

  Serial.println();
  Serial.println("================================");
  Serial.println(" AUTOMATIC WATER TANK SYSTEM");
  Serial.println("================================");
  Serial.println("System Started");
  Serial.println();
}

void loop() {
```

```
int sensorValue = analogRead(WATER_SENSOR_PIN);

int waterLevel = map(sensorValue, 0, 4095, 0, 100);
waterLevel = constrain(waterLevel, 0, 100);

Serial.print("Sensor Value: ");
Serial.print(sensorValue);
Serial.print("    Water Level: ");
Serial.print(waterLevel);
Serial.println("%");

if (waterLevel < LOW_LEVEL) {

  digitalWrite(RELAY_PIN, HIGH);
  digitalWrite(LED_PIN, HIGH);
  digitalWrite(BUZZER_PIN, HIGH);

  Serial.println("WARNING: LOW WATER LEVEL");
  Serial.println("PUMP: ON");
  Serial.println("TANK IS BEING FILLED");

}
else if (waterLevel >= FULL_LEVEL) {

  digitalWrite(RELAY_PIN, LOW);
  digitalWrite(LED_PIN, LOW);
  digitalWrite(BUZZER_PIN, LOW);

  Serial.println("WATER LEVEL: FULL");
  Serial.println("PUMP: OFF");
  Serial.println("TANK IS FULL");

}
else {

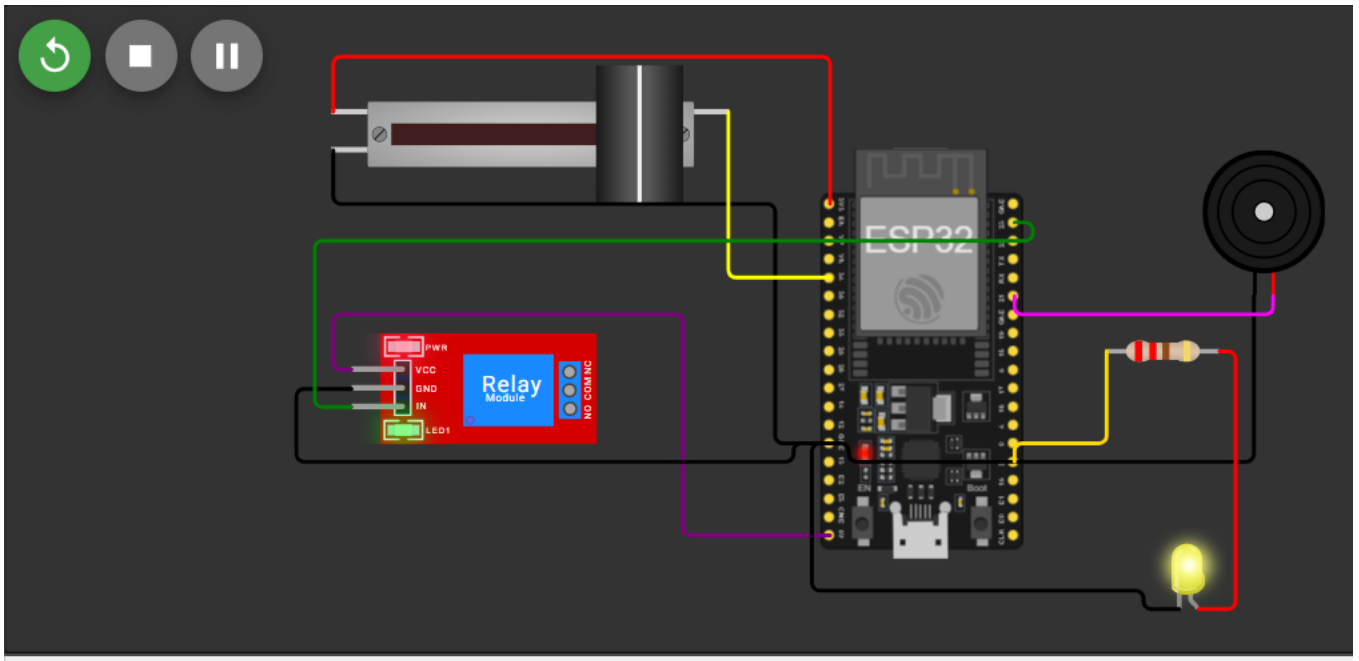
  digitalWrite(RELAY_PIN, HIGH);
  digitalWrite(LED_PIN, HIGH);
  digitalWrite(BUZZER_PIN, LOW);

  Serial.println("WATER LEVEL: NORMAL");
  Serial.println("PUMP: ON");
  Serial.println("TANK IS BEING FILLED");
}
```

```
  Serial.println("----------------------------------");

  delay(1000);
}
```

Output:




```
Sensor Value: 2364      Water Level: 57%
WATER LEVEL: NORMAL
PUMP: ON
TANK IS BEING FILLED
```

Practical No: 09

Aim : Develop an iot module to which measures the intensity of light and send the same to your pc/phone wokwi practical

IoT Practical: Light Intensity Measurement Using Wokwi

Aim: To develop an IoT module that measures the intensity of light and sends the measured data to a PC or mobile phone using the Wokwi simulator.

Components Required

Sr.No	Components	Purpose
1.	ESP32	Microcontroller with Wi-Fi
2.	LDR Sensor	Measures light intensity

Code:

```
#include <WiFi.h>
#include <WebServer.h>

const char* ssid = "Wokwi-GUEST";
const char* password = "";

WebServer server(80);

const int LDR_PIN = 34;

int lightValue = 0;
String lightStatus = "";

void handleRoot() {
  lightValue = analogRead(LDR_PIN);

  if (lightValue < 1000) {
    lightStatus = "Low Light";
  } else if (lightValue < 2500) {
    lightStatus = "Medium Light";
  } else {
    lightStatus = "High Light";
  }

  String page = "<!DOCTYPE html><html><head>";
  page += "<meta name='viewport' content='width=device-width, initial-scale=1'>";
```

```
  page += "<meta http-equiv='refresh' content='2'>";
  page += "<title>Light Intensity Monitor</title></head>";

  page += "<body style='font-family:Arial;text-align:center'>";
  page += "<h1>IoT Light Intensity Monitor</h1>";
  page += "<h2>Light Sensor Reading</h2>";
  page += "<h1>" + String(lightValue) + " / 4095</h1>";
  page += "<h2>Status: " + lightStatus + "</h2>";
  page += "<p>Page refreshes every 2 seconds.</p>";
  page += "</body></html>";

  server.send(200, "text/html", page);
}

void setup() {
  Serial.begin(115200);
  analogReadResolution(12);

  WiFi.begin(ssid, password);

  Serial.print("Connecting to WiFi");

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\nWiFi Connected!");
  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());

  server.on("/", handleRoot);
  server.begin();

  Serial.println("Web server started.");
}

void loop() {
  server.handleClient();

  static unsigned long lastRead = 0;

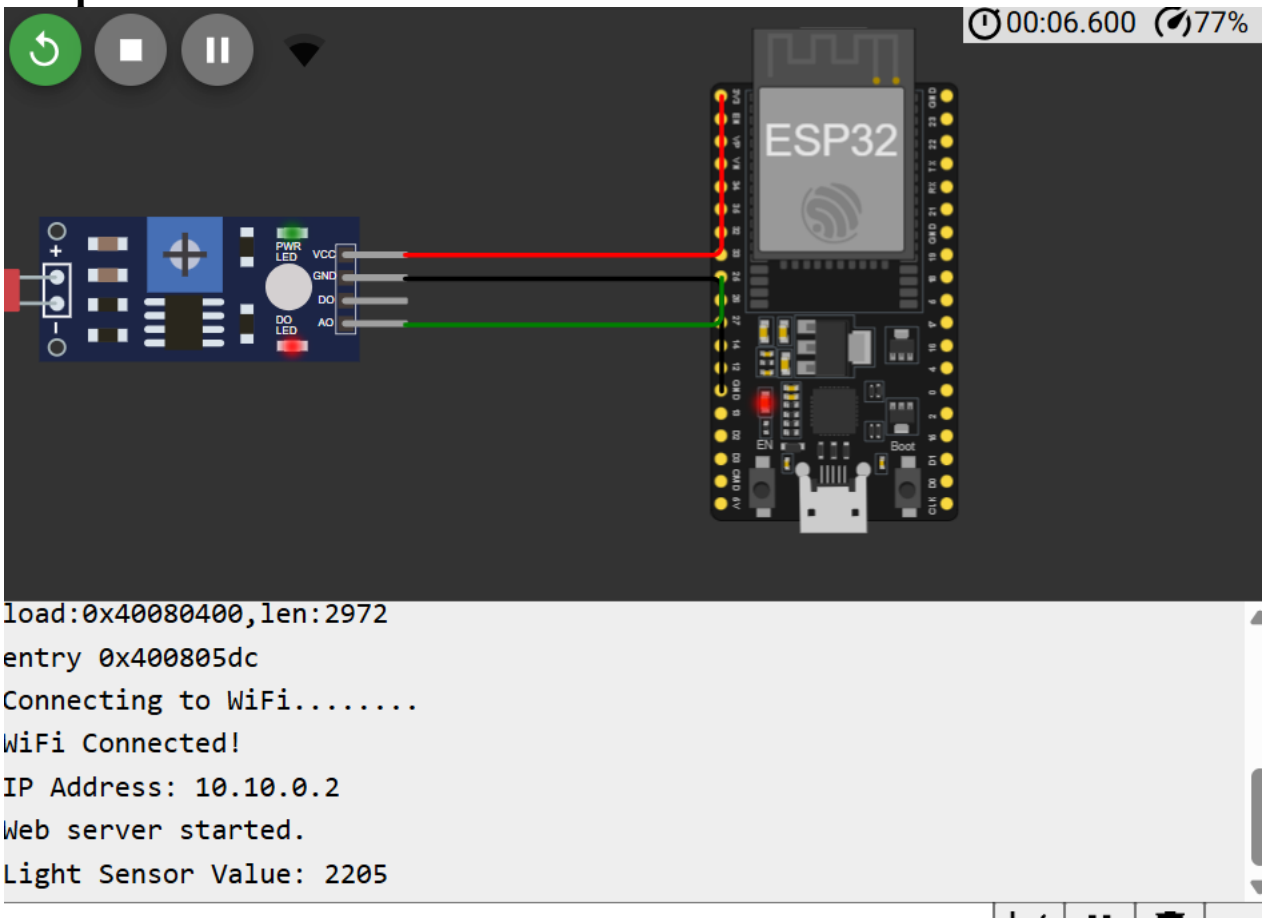
  if (millis() - lastRead >= 2000) {
    lastRead = millis();
```

```
    lightValue = analogRead(LDR_PIN);

    Serial.print("Light Sensor Value: ");

    Serial.println(lightValue);
  }
}
```

Output:



Practical No.: 10

Aim:To develop an IoT-based motion detection application using ESP32 and a PIR sensor that detects human movement and activates an LED and buzzer.

Components Required

Component	Quantity
ESP32 DevKit	1
PIR Motion Sensor (HC-SR501)	1
LED	1
220Ω Resistor	1
Buzzer	1

Circuit Connections

PIR Sensor → ESP32

PIR Pin	ESP32
VCC	5V
GND	GND
OUT	GPIO 27

LED → ESP32

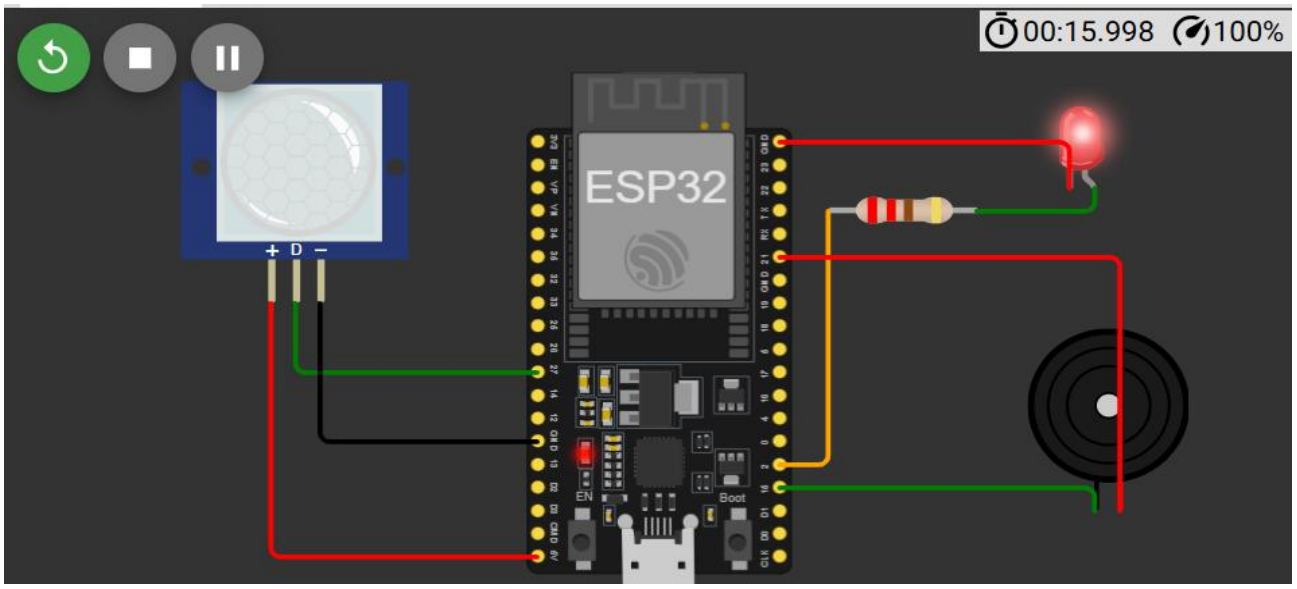
LED	ESP32
Anode (+)	GPIO 2 through 220Ω resistor
Cathode (-)	GND

Buzzer → ESP32

Buzzer	ESP32
Positive (+)	GPIO 15
Negative (-)	GND

Resistor To LED:

Resistor 1 side	ESP 2
Resistor other side	LED 1 A
LED other side	ESP GND



Code:

```
#define PIR_PIN 27
#define LED_PIN 2
#define BUZZER_PIN 15

void setup() {
 Serial.begin(115200);

 pinMode(PIR_PIN, INPUT);
 pinMode(LED_PIN,
 OUTPUT);
 pinMode(BUZZER_PIN,
 OUTPUT);

 digitalWrite(LED_PIN, LOW);
 digitalWrite(BUZZER_PIN, LOW);

 Serial.println("Motion Detection System Started");
 Serial.println("Waiting for motion...");
}

void loop() {

 int motion =
 digitalRead(PIR_PIN); if
 (motion == HIGH) {
  digitalWrite(LED_PIN,
  HIGH);
```

```
  digitalWrite(BUZZER_PIN,
  HIGH);

  Serial.println("Motion Detected!");
  Serial.println("ALARM: Human movement
  detected.");

 }
 else {

  digitalWrite(LED_PIN,
  LOW);
  digitalWrite(BUZZER_PIN,
  LOW);

  Serial.println("No Motion");


 }

 delay(500);
}
```

Output:

```
load:0x40080400,len:2972
entry 0x400805dc
Motion Detection System Started
Waiting for motion...
No Motion
No Motion
No Motion
Motion Detected!
ALARM: Human movement detected.
Motion Detected!
ALARM: Human movement detected.
Motion Detected!
```